Excel Energy

Subscription Platform & Administration Guide

Status: Operations Guide

Version: 3.0.0

Generated: July 2026

1. EXECUTIVE OVERVIEW & ARCHITECTURE

The **Excel Energy Subscription Platform** is a secure, modern, full-stack web application designed for booking and managing energy wellness, aura scanning, and meditation programs. The platform is divided into a client-facing subscriber portal and an administrative dashboard console.

The platform leverages a modern stack combining a high-performance frontend build toolchain with a relational database layer that ensures consistency for transaction registers, invoice histories, check-in records, and message logs.

Core Technology Stack

- **Frontend Layer:** Built on React 19 wrapped in a Vite compiler environment. Employs Redux Toolkit for centralized user authorization and state synchronization, Axios HTTP clients with auto-interceptors to verify JWT signatures, and GSAP (GreenSock Animation Platform) for smooth layout transitions.
- **Backend Layer:** Built using Node.js and Express.js REST API modules. Rate-limiting middlewares protect database triggers, and security headers are enforced via Helmet packages.
- **Database & ORM:** MySQL relational database managed through the Prisma Client engine. Migrations are documented using SQL-equivalent structures.
- **Integrations:** Razorpay webhook bridges for subscription verification and checkout orders, Meta WhatsApp Cloud API gateways for automated notifications, and Nodemailer configurations for email backups.

2. CORE MODULES & INTERFACES

2.1 Exclusive OTP Authentication Flow

To maximize account security and eliminate credential theft, the system utilizes a **passwordless login system** across all portals. Customers, staff practitioners, and administrators log in by entering their registered mobile number, receiving a secure 6-digit OTP code directly to their WhatsApp number, and entering it to unlock access.

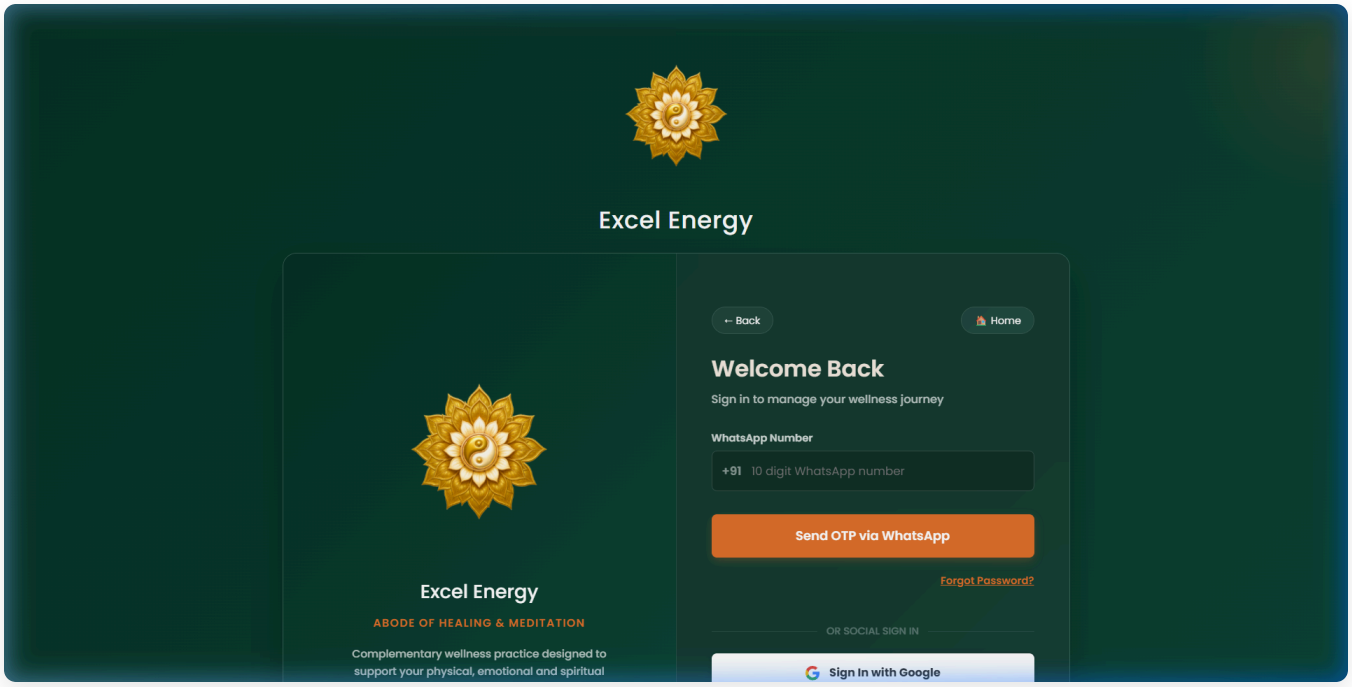

Figure 2.1: Customer login screen featuring exclusive WhatsApp OTP authentication.

2.2 The Employee Portal

Staff practitioners authenticate using the same WhatsApp OTP flow on their custom portal. The system verifies their role as `EMPLOYEE` or `VOLUNTEER` in the DB before granting session access, preventing unauthorized clients or administrators from entering.

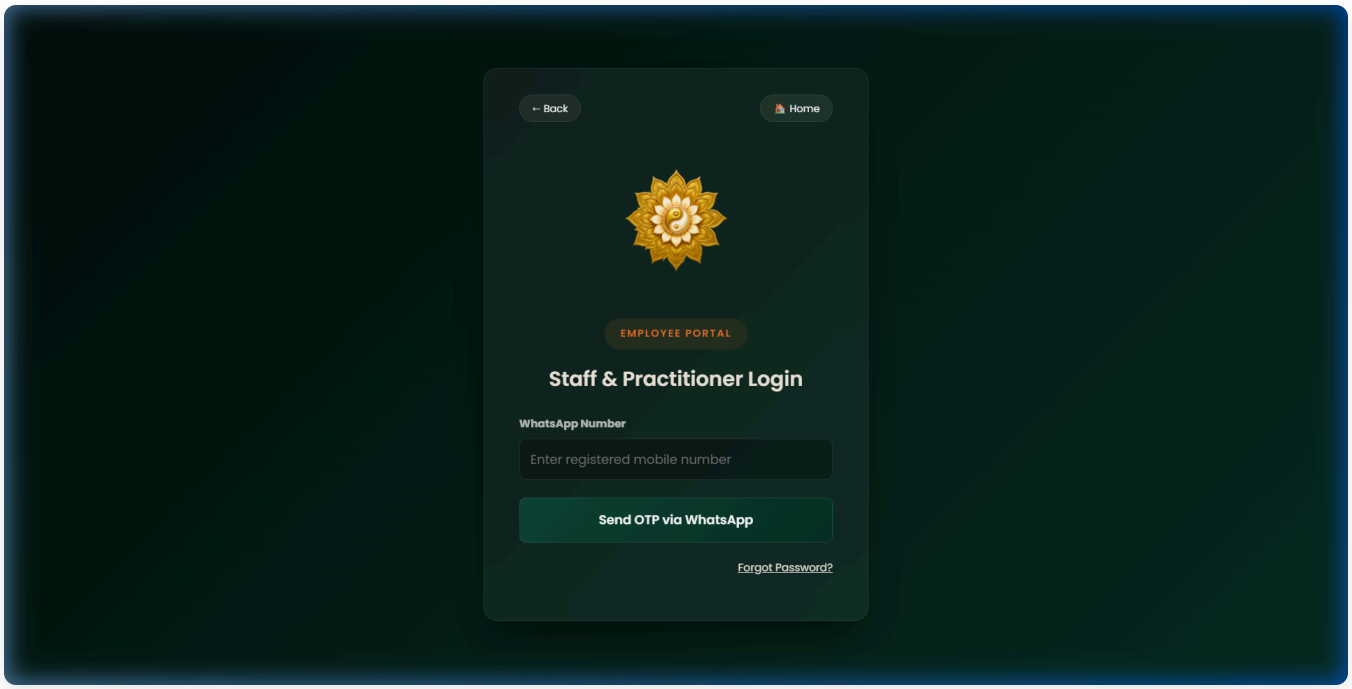

Figure 2.2: Staff Practitioner portal login requiring exclusive WhatsApp OTP.

2.3 The Secure Admin Console

System administrators authenticate using the secure admin route. The login utilizes the WhatsApp OTP flow, ensuring that even if username credentials were compromised, the portal remains fully protected by multi-factor delivery.

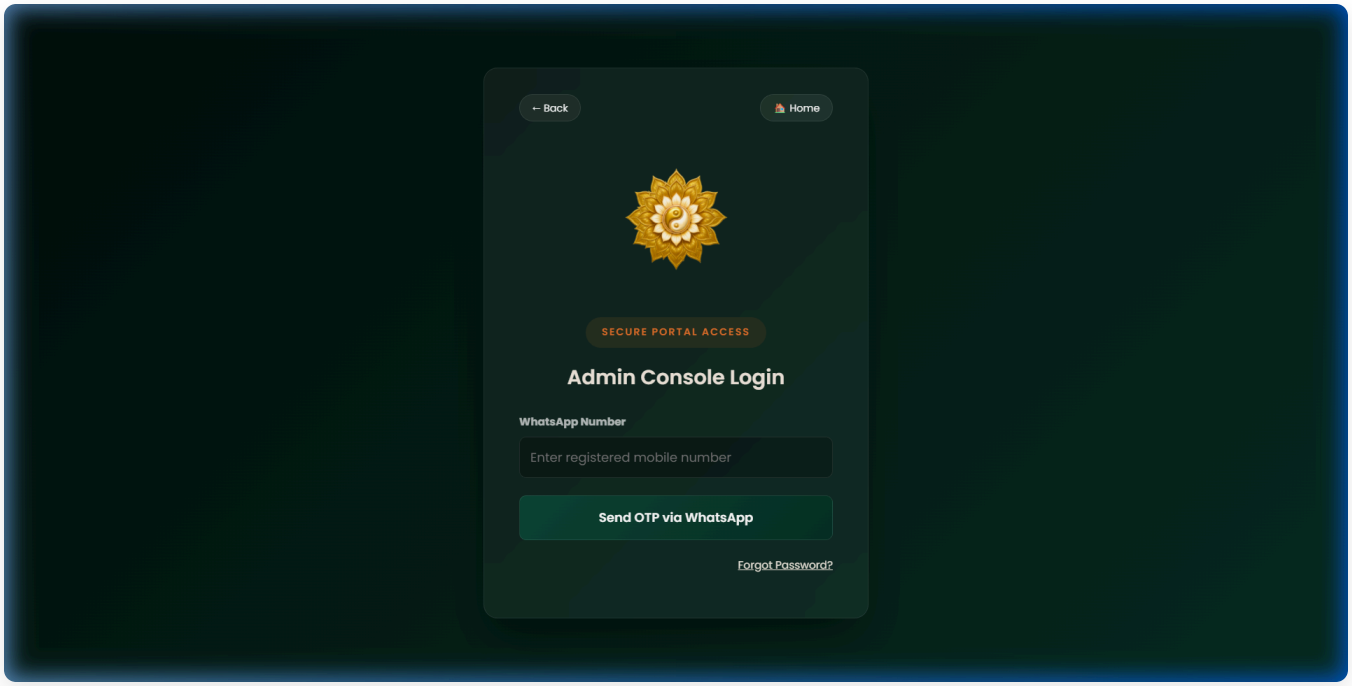

Figure 2.3: Secure Administrator login view requiring OTP code.

2.4 Showcase Gallery Hover Zoom

The *Healing Services Showcase* gallery on the Knowledge and Programs pages renders premium tiles. On hover, the text overlays are cleanly hidden, and the image scales up with full color clarity, displaying the design details baked into the images.

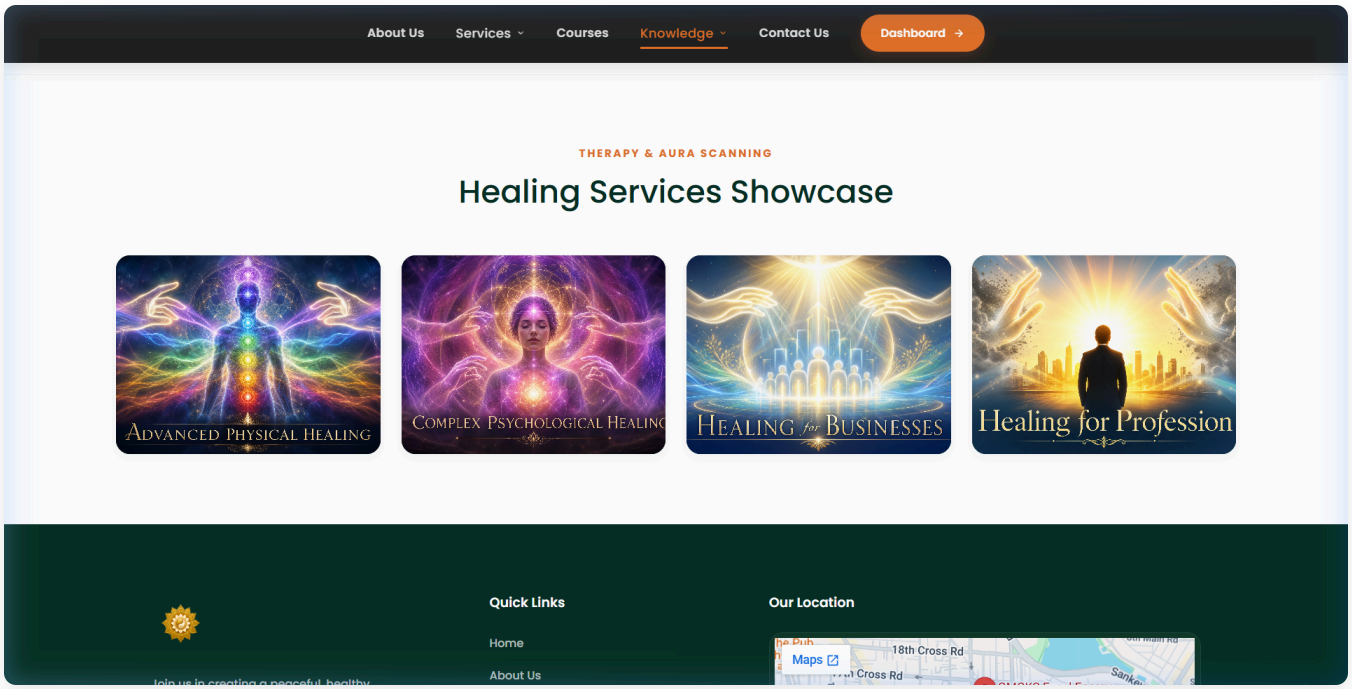

Figure 2.4: Showcase gallery zoom alignment on hover.

3. ADVANCED FEATURES & RECENT ENHANCEMENTS

3.1 Broadcast History Panel & Autofill

To enable admins to reuse previous notifications, we added a **Sent Broadcasts** history panel side-by-side with the announcement creation form. This panel calls a backend `GET /api/admin/broadcasts` route to retrieve the latest 50 announcements from the database.

When an admin clicks on the title of any sent broadcast, the system uses a smart text-parser to split the announcement body. It extracts the raw message description and separates optional Link Labels and Link URLs. These values are autofilled into the form fields, allowing the admin to easily resend or edit templates.

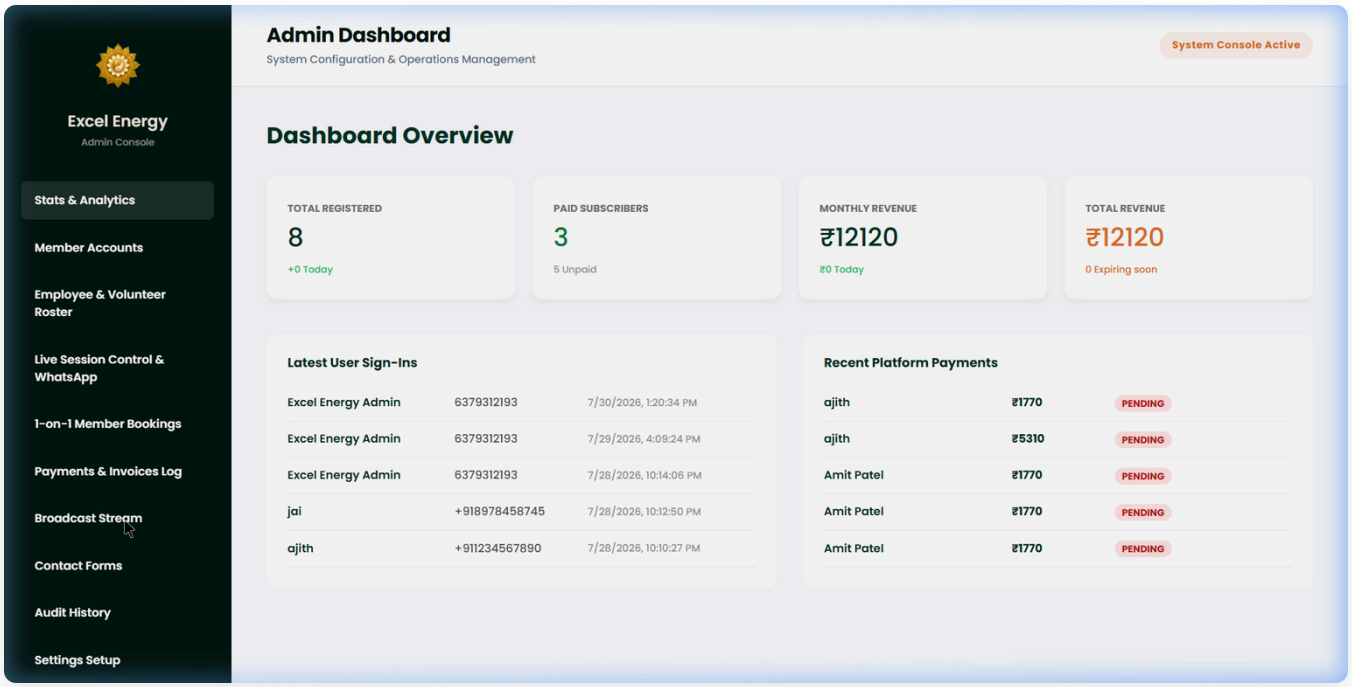

Figure 3.1: Admin Broadcast Tab showing the 3-column layout and the Sent Broadcasts history panel.

3.2 Unified Forgot Password Workflow

We integrated a **Forgot Password** link and verification flow on the Employee Login and Secure Admin Login pages. If clicked, the form changes to collect the user's registered WhatsApp number,

requests a 6-digit OTP code, and lets the user update their password hash securely.

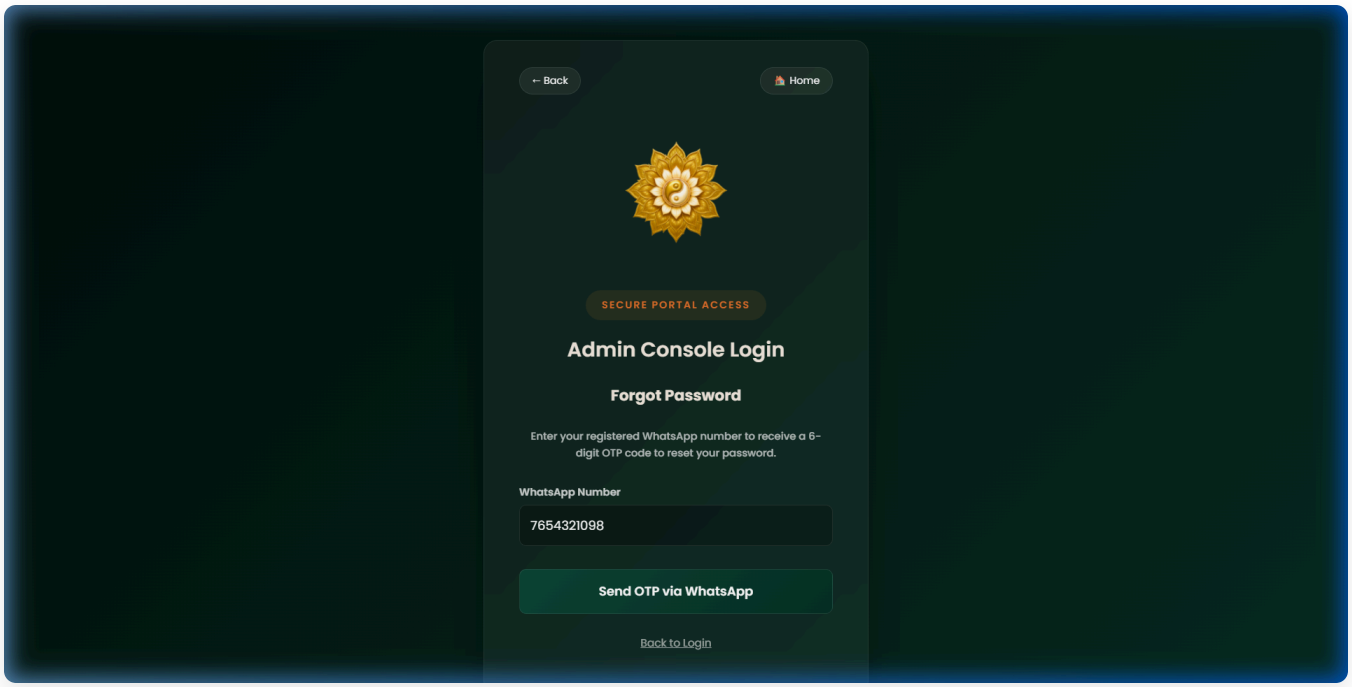

Figure 3.2: Frosted-glass panel displaying the OTP verification stage of the password reset flow.

4. DATABASE SCHEMA & REST API

4.1 Core Prisma Schema Models

The platform maps relationships through schema definitions. Below are the key tables defined in `schema.prisma`:

```
model User {
  id           Int      @id @default(autoincrement())
  username     String   @unique
  name         String
  phone        String   @unique
  email        String?
  passwordHash String?
  status       String   @default("ACTIVE")
  roleId       Int
  createdAt    DateTime @default(now())

  subscriptions     Subscription[]
  payments          Payment[]
  userNotifications UserNotification[]
}

model Notification {
  id             Int       @id @default(autoincrement())
  title          String
  description    String    @db.Text
  targetAudience String    @default("ALL") // ALL, PAID, UNPAID
  createdAt      DateTime  @default(now())
}
```

4.2 Key REST API Routing

Endpoint	Method	Auth Required	Description
`/api/auth/request-otp`	POST	No	Request verification code for user registration or OTP login.
`/api/auth/login-otp`	POST	No	Verifies OTP code and authenticates session access for all user roles.
`/api/auth/forgot-password-request`	POST	No	Dispatches verification code to registered phone numbers.
`/api/auth/forgot-password-reset`	POST	No	Resets password hash if the OTP code matches and is valid.
`/api/admin/broadcasts`	GET	Yes (Admin)	Retrieves previous bulk notifications to show in the history panel.

5. DEPLOYMENT & LOCAL INSTRUCTIONS

Local Environment Setup

To run the application locally on a development machine:

```
# 1. Start the Backend API
cd backend
npm install
cp .env.example .env     # Update DATABASE_URL with MySQL credentials
npx prisma db push       # Sync tables
npx prisma db seed       # Seed initial users & roles
npm run dev              # Starts Express on http://localhost:5000

# 2. Start the Frontend Client
cd ..
npm install
npm run dev              # Starts React Vite on http://localhost:5173
```

Production Deployment Roster

- **Frontend Hosting:** Deploy the client package to Vercel or Netlify. Provide `VITE_API_URL` pointing to the backend domain.
- **Backend VPS:** Deploy Node to a Linux VPS (Ubuntu) running PM2 daemon processes. Run Nginx reverse proxies with automatic Certbot Let's Encrypt SSL certifications.

> **IMPORTANT SECURITY NOTE**

> Ensure JWT secrets and database connection links are loaded exclusively from system environment variables. Never commit plaintext environment keys or database credentials to git source trees.